A smaller inference interface for intuitionistic primitive recursive arithmetic

Deterministic witness opening and measured checker reductions

Prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

An equality-bearing witness elimination can determine its replacement from one fixed instance of the formula being opened. Specializing the opened clause at that value leaves the outer goal unchanged and removes an explicit replacement term, an equality-proof premise, and an intermediate formula from the certificate interface. The resulting presentation has three rule families and one fixed reflexivity clause. A bootstrap through restricted openings and a strong substitution theorem proves equality of the entire canonical clause consequence relation with the preceding presentation. It therefore preserves full first-order iPRA bi-interpretability, whole domains, actual equality, and both identity composites. Its matched complete checker decreases from 2,273 to 2,253 normalized Python bytes. Independently, a proved continuation refactor of the existing compact calculus decreases its complete byte profile from 516 to 508 bytes and its separate AST profile from 278 to 276 nodes. The new mathematical presentation is not the one implemented by the 508-byte checker. Global minimality and a single-program improvement in every size coordinate are not claimed.

Contents

1 Results and the meaning of simpler **3**

1.1 The inspected comparison . . . 3

1.2 Verification status . . . 4

2 Opening a witness at the value determined by its clause **4**

2.1 The complete presentation . . . 4

2.2 Direct soundness, including self-containing targets . . . 6

2.3 The bootstrap must precede ordinary elimination . . . 6

2.4 A full numerical substitution theorem . . . 7

2.5 Equality transport from a determined witness . . . 9

2.6 Exact comparison with the predecessor . . . 10

2.7 What has been simplified and what remains trusted . . . 11

3 The computation built into the syntax **11**

3.1 Termination, confluence, and substitution . . . 12

3.2 Why normalization does not strengthen the arithmetic . . . 12
3.3 Uniform resource bounds . . . 13

4 Full first-order iPRA and both interpretation composites 13
4.1 The target theory . . . 13
4.2 An intermediate ordinary scalar theory . . . 14
4.3 All ordinary logical operations are available . . . 14
4.4 The numerical interpretations . . . 15
4.5 Whole domains and return translations . . . 15

5 The executable presentation and its checks 16
5.1 Exact certificate interface . . . 16
5.2 Measured effect of the new rule . . . 17
5.3 Executed evidence . . . 17
5.4 Reproduction . . . 18

6 A separate, measured implementation reduction 18

7 Further research questions 20

A Complete selected checker 20

B The separate 508-byte compact profile 22

1 Results and the meaning of simpler

This report gives a further mathematical restriction of the inference interface for full first-order intuitionistic primitive recursive arithmetic. The principal rule is *deterministic witness opening*. An opened formula supplies, when it has the prescribed form, its own equality for specializing the witness. A proof certificate no longer supplies a replacement term, a proof of the witness equality, or an intermediate branch formula. Only the major proof and a proof of the unchanged outer goal are supplied.

The construction has three inference families, Use, Introduce, and Open, and one explicit universal-reflexivity axiom clause. Its terms are full computation normal forms. Both normalization and the deterministic witness calculation belong to the trusted specification and to the measured checker. The theory proves exactly the same canonical clause sequents as the preceding presentation. Its ordinary realization is therefore bi-interpretable and equiconsistent with full first-order iPRA, with whole numerical domains, actual equality, and numerical composite maps having graph $x = y$.

Two distinct improvements. The mathematical rule restriction and the smallest-program result are different results. The former has a complete, readable checker of **2,253 normalized bytes**, compared with 2,273 bytes for its immediate predecessor. The latter refactors the existing compact calculus and reduces its complete checker from **516 to 508 normalized bytes**. The 508-byte program implements the inherited compact calculus; it is not a checker for the new three-rule presentation.

Complete module	Bytes	Tokens	AST nodes
Witness-discharge predecessor	2,273	732	813
Deterministic witness opening	2,253	730	809
Inherited compact byte profile	516	265	306
Continuation refactor, byte profile	508	267	302
Inherited AST profile	602	260	278
Continuation refactor, AST profile	580	263	276
Inherited native goal validator	788	358	413
Continuation refactor, native validator	780	360	409

The first pair isolates the mathematical interface change in the supplied goal-directed implementation. It saves 20 bytes (about 0.88%), two significant tokens, and four AST nodes. The mathematical comparison therefore improves all three coordinates in this matched implementation. The compact byte profile saves 8 bytes (about 1.55%), while using two more tokens. The observed corpus minimum of 256 significant tokens is not improved. No single program dominates every old coordinate. These are attained upper bounds, not mathematical lower bounds on all possible presentations. An absolute simplest axiomatization is not established here.

1.1 The inspected comparison

The comparison is pinned to ProveIt commit

`17898ea7e6a1300dfdcad71040d5daeed67916d4`.

The inspected project is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. Its README lists 67 filed research packages and a Lean development.[1] The comparison also includes all six ZIP archives in its `docs/incoming` directory at that commit: `clause_contexts`, `clause_guards`, `mirror_core`, `seed_evaluation`, `uniform_binding`, and `witness_discharge`. The last is the immediate mathematical predecessor used here.[2] Source and archive identities are recorded in the accompanying provenance file.

Several ingredients are inherited and are not claimed as new: the scalar iterator, the paired binder, the clause representation, the fixed reflexivity clause, full normalization, the two-row equality package, and the whole-domain arithmetic interpretation. The new restriction is to determine the witness from a fixed probe and change the opened clause accordingly. The proof that this restricted rule still suffices needs a substitution theorem; simply reusing a branch under a different clause would be invalid.

1.2 Verification status

The mathematical equivalence in this report is a written proof. The archive also contains standalone checkers, executable proof examples, negative controls, source measurements, and independent audit notes. These tests check the implementation; they do not establish the universal mathematical theorems by finite sampling. The new rule and Python programs have not been verified in Lean or Rocq. The inherited Lean bridge is identified by its actual theorem names and pinned source, rather than being presented as a newly rebuilt formalization.

2 Opening a witness at the value determined by its clause

The preceding witness-discharge calculus asks for three proofs at an elimination: the paired formula, an equality identifying its fresh witness, and a formula to be exported after substitution. It also asks for the substitution target and the formula before substitution. The presentation below removes the equality proof, target, and pre-substitution formula from that interface. A fixed probe of the paired formula determines the value used in the opened clause. The branch then proves the original outer goal without changing it.

The fresh numerical parameter is retained in *every* branch. In particular, the selected value is allowed to contain that parameter. This choice avoids an additional scope restriction on the selector and is essential to the exact rule stated here.

2.1 The complete presentation

Terms are full normal forms of

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h),$$

under the context-closed iterator reductions

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow h[I(n, a; u.h)/u].$$

Only u is bound in h. Write N for normalization. All numerical substitutions avoid capture and are followed by N on numerical subterms. Normalization leaves the formula constructors unchanged.

The formulas are equations and $D_{x,y}B$; their ordinary readings are actual equality and

$$(D_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\big).$$

Let $H = (0=0)$, $1 = S0$, and $2 = S(S0)$.

A context is a finite list of clauses $[\vec{A}; y.B(y)]$, each having zero or one antecedent. Its bound variable y is absent from the antecedent. Its reading is $\forall y\, B(y)$ for an empty antecedent list, and $A \to \forall y\, B(y)$ otherwise. An ordinary assumption F is the unconditional constant clause $[\,; v.F]$. The initial context contains exactly the following fixed axiom clause:

$$\mathcal{R} = [\,; y.y = y]. \tag{Ax}$$

This explicitly accounts for reflexivity. The designated contradiction is $0 = S0$.

For a matrix $B(x,y)$ and a fresh parameter b, compute

$$B_0 = N(B(b,0)), \qquad B_1 = N(B(b,1)),$$

and define

$$\rho(B,b) = \begin{cases} t, & B_0 \equiv (0=0) \text{ and } B_1 \equiv (b=t), \\ b, & \text{otherwise.} \end{cases} \tag{Selector}$$

Here $\equiv$ denotes literal equality of canonical syntax. The left side in the successful probe must be exactly the fresh parameter b. The right side t may contain b. No theorem search, equality decision up to provability, certificate choice, or additional equality proof is hidden in the selector.

For any well-scoped term r of the branch, abbreviate the canonical clause

$$\mathcal{C}_B(r) = [[N(B(r,0))]; y.N(B(r,y))].$$

Substituting r into this clause is capture avoiding and simultaneous. An occurrence of b inside r is not substituted again.

The proposed presentation AO has exactly three instruction families.

1. **Use.** For a retained clause $[\vec{A}; y.B] \in \Gamma$, prove its one antecedent when present and infer $N(B(t))$, at any well-scoped numerical term t. An unconditional clause has no proof premise.

2. **Introduce.** From
$$\Gamma, B(t,\eta) \vdash B(t,Sz) \quad \text{infer} \quad \Gamma \vdash D_{x,y}B, \tag{I}$$
where z is fresh. Set $\eta = z$ exactly when B has equation root and the canonical witness t is literally 0; otherwise set $\eta = 0$. The displayed branch assumption is an unconditional constant clause.

3. **Open.** With b fresh, infer
$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, \mathcal{C}_B(\rho(B,b)) \vdash A}{\Gamma \vdash A}. \tag{O}$$
The branch retains the fresh parameter b, whether or not the selector returns it. The conclusion A, the retained context Γ, and the passive parameters of the major formula are independent of b.

The branch of O proves the same outer goal A. There is no separate formula F to choose, no substitution in the exported conclusion, no equality-proof premise, and no supplied replacement term.

These are all the primitive inferences. Full normalization is part of the trusted syntax and checker. There is no additional equality-transport, computation, numerical-case, induction, weakening, cut, or ordinary paired- elimination rule. The axiomatic source of reflexivity remains the fixed clause (Ax).

2.2 Direct soundness, including self-containing targets

Lemma 2.1 (The original opened clause determines the selected equality)**.** *In the context $\Gamma, \mathcal{C}_B(b)$, the equation $b = \rho(B, b)$ is derivable by Use and the fixed reflexivity clause.*

Proof. If the selector succeeds, the clause's antecedent is literally H. Use $\mathcal{R}$ at 0 and then use $\mathcal{C}_B(b)$ at 1; this gives $b = t$. If the selector fails, use $\mathcal{R}$ at b. □

Theorem 2.2 (Soundness of automatic opening)**.** *Rule O is sound in intuitionistic first-order equality logic over the ordinary scalar arithmetic, even when $\rho(B, b)$ contains b.*

Proof. Eliminate the existential in the ordinary reading of the major formula at a fresh parameter b. Its actual local assumption is $\mathcal{C}_B(b)^\circ$. Lemma 2.1 supplies $b = \rho(B, b)$. Ordinary equality substitution in the fixed clause template $\mathcal{C}_B(v)^\circ$ gives $\mathcal{C}_B(\rho(B, b))^\circ$. The supplied branch then proves A. Since A and the retained outer assumptions are independent of b, ordinary existential elimination discharges the original witness.

No occurs check is needed on the selected target. Equality substitution from $b = t(b)$ to a fixed formula instance $C(t(b))$ is valid; it is a single simultaneous substitution. The fresh b remains in the branch, including in the selected clause when necessary, and the final goal does not contain it. The rule does not assert that the equation $b = t(b)$ has a solution independently of the major premise. □

2.3 The bootstrap must precede ordinary elimination

The new branch assumption is generally $\mathcal{C}_B(\rho(B, b))$, whereas ordinary paired elimination expects $\mathcal{C}_B(b)$. Thus ordinary elimination cannot be justified merely by observing that its goal is b-free. We first establish the numerical substitution theorem needed to change its branch assumption. This order avoids assuming the desired equality or elimination principle in its own proof.

Weakening, injective fresh renaming, and cut of an ordinary assumption are admissible by direct induction on derivations. For O, these operations leave its matrix and determined target unchanged up to fresh renaming. For cut, replace uses of the unconditional constant clause for the cut assumption by its supplied proof; freshen local parameters before insertion. No arbitrary numerical substitution theorem is used in this argument.

Lemma 2.3 (Openings on which the selector necessarily falls back)**.** *The selector returns its fresh parameter in each of the following cases:*

1. *the matrix B does not use its witness coordinate x;*
2. *the matrix B has outer constructor D.*

Consequently, O is literally ordinary paired elimination on these matrices.

Proof. In the first case the normalized probe at 1 does not contain the fresh parameter b, so it cannot be an equation whose left side is b. In the second case the probe is still a D formula, because numerical normalization never removes a formula constructor. Neither can satisfy the successful guard in (Selector). □

For unused w write $q_y A(y) = D_{w,y} A(y)$. Its eliminations fall under the first case of Lemma 2.3. Therefore, before any general equality reasoning, we have

$$\frac{\Gamma, A(0) \vdash A(Sz)}{\Gamma \vdash q_y A(y)}, \qquad \frac{\Gamma \vdash q_y A(y) \qquad \Gamma \vdash A(0)}{\Gamma \vdash A(t)}. \tag{Q}$$

The first rule uses witness 1 in I, which forces case mode. The second uses O and then Use on the opened clause $[A(0); y.A(y)]$. As a consequence, proofs of $A(0)$ and $A(Sz)$ for fresh z yield a proof of $A(t)$ at every well-scoped term t. These numerical cases use only fallback openings.

Lemma 2.4 (All case introductions)**.** *For every matrix B and every canonical witness t, the inference*

$$\Gamma, B(t,0) \vdash B(t,Sz) \quad \Longrightarrow \quad \Gamma \vdash D_{x,y} B$$

is derivable in AO*. Its derivation uses only fallback openings, in addition to Use and I.*

Proof. Only the case of an equation matrix $E(x,y)$ and witness $t \equiv 0$ is missing from the primitive rule. Define the normalizing selector $C(v,a,c) = N(I(v,a;u.c))$, with u absent from c, and the equation gate

$$G_v(s = t) = (C(v,0,s) = C(v,0,t)), \qquad L(E) = q_v G_v E.$$

The literal normalized endpoints are $G_0 E \equiv H$ and $G_{Sw} E \equiv E$. The rules (Q) and reflexivity therefore give uniform derivations $E \vdash L(E)$ and $L(E) \vdash E$. Every opening in these derivations has an unused witness coordinate.

Suppose the given derivation is $\Gamma, E(0,0) \vdash E(0,Sz)$. Set $A(x,y) = L(E(x,y))$. Under $A(0,0)$, unwrap the equation, insert the given derivation by ordinary- assumption cut, and wrap the resulting equation. This gives $A(0,Sz)$. Since A has a D root, I at witness 0 uses cases and gives $D_{x,y} A(x,y)$.

Open this formula at a fresh c. Its matrix is compound, so this is a fallback opening by Lemma 2.3. It supplies the ordinary clause $[A(c,0); y.A(c,y)]$. To introduce $D_{x,y} E(x,y)$ at witness c, assume $E(c,0)$, wrap it as $A(c,0)$, use that clause at Sz, and unwrap $A(c,Sz)$ as $E(c,Sz)$. The witness c is not literal zero, so I uses cases. Close I and the fallback opening. The result is the required $D_{x,y} E$. Neither general equality transport nor general numerical substitution in derivations has been used. □

2.4 A full numerical substitution theorem

Theorem 2.5 (Normalized substitution in derivations)**.** *For every finite derivation $\Gamma \vdash_{\mathsf{AO}} A$ and every well-scoped simultaneous substitution σ for its free numerical parameters, there is an effectively constructed derivation*

$$N(\Gamma\sigma) \vdash_{\mathsf{AO}} N(A\sigma).$$

The fixed clause $\mathcal{R}$ is unchanged. The theorem permits a source parameter to be mapped to a term containing a receiving parameter with the same printed name; substitution is simultaneous, not iterative.

Proof. Induct on the finite derivation, with the substitution universally quantified in the induction hypothesis. Freshen all bound and local parameters before applying a substitution. Use is immediate from normalization–substitution coherence. Reflexivity uses the same fixed axiom clause.

For I, apply the induction hypothesis to its strictly smaller branch, extending σ by a fresh parameter for its local variable z. If the mode is preserved, the resulting rule instance is direct. An old progressive instance has a literal zero witness, which stays zero; the equation root also stays an equation. The only possible change is from cases to progress when a previously nonzero witness normalizes to zero and the matrix is an equation. In this event use the already proved all-case rule, Lemma 2.4, on the substituted branch. This is an insertion of a known derived inference, not an appeal to the substitution theorem on a larger newly constructed proof.

It remains to handle O. To distinguish source and receiving scopes, call its old fresh parameter b and its new fresh parameter c. Translate its strictly smaller major proof under σ, and compute

$$B' = N(B\sigma), \qquad r' = \rho(B', c).$$

There are two cases according to the *old* guard.

If the old guard succeeded, write its old target as $t(b)$. Apply the induction hypothesis to the old branch with σ on outer parameters and $b \mapsto c$. Success is preserved under this protected substitution: the old base H stays H, and the old probe $b = t(b)$ becomes $c = N(t\sigma[b \mapsto c])$. Thus the new guard also succeeds and

$$r' = N(t\sigma[b \mapsto c]).$$

By coherence, the substituted branch clause is exactly $\mathcal{C}_{B'}(r')$. This argument does not require t to be independent of b.

If the old guard failed, its old branch clause is $\mathcal{C}_B(b)$. Apply the induction hypothesis to that old branch with the substitution that extends the outer σ by

$$b \longmapsto r'.$$

This is allowed whether $r' = c$ or r' contains c in a larger term. The source parameter b is replaced in one simultaneous pass. Its clause becomes exactly $\mathcal{C}_{B'}(r')$, again by coherence. The branch goal and retained outer context did not contain b, so their images are precisely the required $N(A\sigma)$ and $N(\Gamma\sigma)$. This covers a guard that stays false and a guard newly made true by normalization after substitution.

In either case O applies to the translated major and branch, proving $N(A\sigma)$. Every recursive call used the induction hypothesis for a strictly smaller original proof, including when the chosen replacement contains the new fresh parameter. The construction is therefore well founded and yields a finite proof. □

The selector's successful condition is preserved under protected passive- parameter substitution, but its failed condition need not be. The second case of this proof is necessary: treating the selector as an invariant Boolean test would leave a gap.

Corollary 2.6 (Ordinary paired elimination)**.** *For arbitrary matrices, not just those of Lemma 2.3, the rule*

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, \mathcal{C}_B(b) \vdash A}{\Gamma \vdash A}, \qquad b \notin \mathrm{FV}(\Gamma, A),$$

is derivable in AO*.*

Proof. Compute $r = \rho(B, b)$ in the receiving fresh-parameter scope. Apply Theorem 2.5 to the supplied branch, leaving its outer parameters fixed and replacing its source fresh parameter by r. Its assumption becomes $\mathcal{C}_B(r)$. Its goal remains A, because A has no free source parameter. The receiving scope retains b, so this is legitimate even if r contains b. The new rule O now applies. If the selector falls back, this is just a fresh renaming or the identity instance of the same argument. □

2.5 Equality transport from a determined witness

The numerical selector and parity terms

$$C(v, a, c) = N(I(v, a; u.c)), \qquad P(y) = N(I(y, 0; u.C(u, 1, 0)))$$

satisfy the literal canonical identities

$$C(0, a, c) \equiv a, \quad C(Sv, a, c) \equiv c, \quad P(1) \equiv 1, \quad P(2) \equiv 0.$$

No range theorem or induction for parity is needed below.

Theorem 2.7 (Uncrossed transport)**.** *For every fixed numerical term template $h(w)$, the inference*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash N(h(s)) = r}{\Gamma \vdash N(h(t)) = r}$$

is derivable in AO*.*

Proof. Choose fresh x, y and use the two-row matrix

$$B(x, y) = \left(C(y, 0, C(P(y), N(h(x)), x)) = C(y, 0, C(P(y), r, t))\right). \tag{Rows}$$

Its relevant normalized instances are

$$B(x, 0) \equiv H, \qquad B(x, 1) \equiv (x = t), \qquad B(x, 2) \equiv (N(h(x)) = r).$$

Introduce $D_{x,y}B$ at witness s. Its successor goal is

$$C(P(Sz), N(h(s)), s) = C(P(Sz), r, t).$$

Use the numerical-case construction (Q) on

$$A(v) = \left(C(v, N(h(s)), s) = C(v, r, t)\right)$$

at target $P(Sz)$. The zero instance is the supplied $N(h(s)) = r$, and every successor instance is the supplied $s = t$. Choose the case eigenparameter fresh independently of z. This construction ignores I's local assumption, so it works in either mode of I, including when s is literal zero.

Open the resulting D formula. Its base is H and its fixed probe at 1 is $b = t$, so its selected value is the original outer term t. The branch therefore receives $\mathcal{C}_B(t)$ directly. Use this clause at 2, with the reflexive antecedent H. Its conclusion is $N(h(t)) = r$, which is the unchanged outer goal. Rule O finishes. There is no equality-proof branch and no substitution in the exported formula. All equality use has been obtained through the determined witness in the clause. □

The order of these two rows puts the equality identifying the witness at the smaller fixed probe 1. The other row is read by ordinary Use at 2. The naming method is inherited; the new inference determines and uses its witness automatically.

Corollary 2.8 (Full equality)**.** *Symmetry, transitivity, term congruence, crossed term transport, and transport in every Eq/D formula template are derivable.*

Proof. In Theorem 2.7, take the identity template and the reflexive minor $s = s$ to derive $t = s$ from $s = t$. This gives symmetry. Taking a reflexive minor for a general template gives congruence, and the identity template together with symmetry gives transitivity. Symmetry after uncrossed transport gives crossed transport. Applying term transport successively to both sides transports an equation template.

For a compound template $D_{x,y}E(v, x, y)$, induct externally on formula depth. From $s = t$ and $D_{x,y}E(s, x, y)$, use the derived ordinary paired elimination at a fresh c. Introduce $D_{x,y}E(t, x, y)$ at witness c, which forces case mode. Under $E(t, c, 0)$, transport backward at the strictly smaller matrix using $t = s$, obtaining $E(s, c, 0)$. Use the old opened clause at Sz, then transport the result forward at the smaller matrix to $E(t, c, Sz)$. Close I and ordinary elimination. This is induction on the finite formula syntax, not induction inside the arithmetic theory on quantified formulas. □

2.6 Exact comparison with the predecessor

Let WD be the predecessor with the same canonical syntax, clauses, fixed axiom $\mathcal{R}$, Use, and I. Its equality-aware discharge has a major $D_{x,y}B$, opens $\mathcal{C}_B(b)$, asks for branch proofs $b = t$ and F, and exports $N(F[t/b])$ when it is independent of b.

Theorem 2.9 (The entire consequence relation is unchanged)**.** *For every canonical formula A and every finite canonical context Δ of clauses with zero or one antecedent,*

$$\mathcal{R}, \Delta \vdash_{\mathsf{AO}} A \quad \Longleftrightarrow \quad \mathcal{R}, \Delta \vdash_{\mathsf{WD}} A.$$

Both directions have effective finite proof transformations, with the end context and end formula left unchanged.

Proof. First translate a WD proof recursively into AO. Use and I are shared. At a WD discharge, the translated original branch proofs give $b = t$ and F under $\mathcal{C}_B(b)$. Full formula transport, Corollary 2.8, gives $N(F[t/b])$ there. That result was required by WD to be independent of b, so derived ordinary paired elimination, Corollary 2.6, exports it. The lemmas used here were proved in AO before this translation and without a translation of the node currently being replaced. There is no circular appeal to an omitted WD rule.

Conversely, translate an AO proof recursively into WD. Use and I are again shared. For its O node, WD has ordinary paired elimination and all ordinary equality reasoning. Open its major at b, obtaining $\mathcal{C}_B(b)$. If the selector falls back, the translated AO branch already has the required assumption, so ordinary elimination finishes directly.

If the selector succeeds with target $t(b)$, Use and reflexivity give $b = t(b)$. The base of $\mathcal{C}_B(b)$ is H, so for every fresh z its clause gives $B(b, Sz)$. Transport in the witness coordinate using $b = t(b)$ gives $B(t(b), Sz)$. Case introduction at the unused witness 1 proves the dummy-witness closure $q_y B(t(b), y)$, ignoring its temporary zero-base assumption. Ordinary elimination of this closure supplies exactly the desired clause $\mathcal{C}_B(t(b))$. The recursively translated AO branch can now be used, weakened by the other retained assumptions and freshened at the unused dummy witness. It proves A. Close the dummy-witness elimination and the original paired elimination. The final goal is independent of b, as required by O.

This construction remains valid if t contains b: throughout the inner construction b is an available outer parameter. Only the final ordinary elimination discharges it. Each replacement is a finite derivation built from already established WD rules. Freshen the finitely many local parameters before insertion. □

Composing this identity-language equivalence with the predecessor's canonical/raw comparison and the native/scalar/iPRA bridge yields bi-interpretability with full first-order intuitionistic primitive recursive arithmetic. Both numerical domains are whole, equality is actual equality, and both numerical composite isomorphisms retain the graph $x = y$. The two formula round trips are preserved because this comparison changes neither formula syntax nor ordinary readings. The preserved contradiction $0 = S0$ and the two effective proof transformations also give equiconsistency. This conclusion is not inferred from standard-natural-number agreement.

2.7 What has been simplified and what remains trusted

Both presentations have three instruction families and the same fixed reflexivity axiom. The new O instruction has two proof premises, compared with the predecessor's three. It removes the supplied equality target, the separate equality-proof subtree, and the supplied branch formula before export. Its branch proves the unchanged goal. In exchange, the kernel computes two fixed matrix instances, tests the literal guard, and substitutes the determined value into its added clause. Whole-checker measurements must count these operations and the unchanged full normalizer. A smaller certificate interface is a mathematical reduction, but it does not by itself establish a repository-wide minimum in program size.

The normalizer is a metasyntactic algorithm, not an object-language universal evaluator. No uniform primitive-recursive normalization bound is asserted. A fuel-bounded certificate checker can charge every normalization and recursive verification step, including both selector probes and the selected-clause substitution, to one supplied fuel account. Each accepted finite proof has some sufficiently large finite fuel value, so the bounded interface presents the same theorems. This adds no arithmetic totality axiom and changes neither interpretation domain.

The new contribution is the determined opening rule and its complete bootstrap: fallback openings suffice for numerical cases and the zero-witness repair; these support full substitution of derivations; substitution recovers ordinary paired elimination; the two-row package recovers all equality; and the entire consequence relation follows. The scalar iterator, normal form syntax, paired binder, clause interface, fixed reflexivity axiom, naming package, and arithmetic bridge are inherited. The universal arguments above are written proofs, rather than a new Lean or Rocq verification. Finite tests of a checker would provide implementation evidence, not replace either interpretation proof.

3 The computation built into the syntax

The new opening rule operates on canonical terms, so normalization is part of the theory's presentation. It is not a cost-free external oracle. This section records the inherited metatheory on which the presentation depends, including the point at which uniform primitive recursion would be an unjustified claim.[3]

3.1 Termination, confluence, and substitution

Close the reductions

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow h[I(n, a; u.h)/u]$$

under every numerical term context, including update bodies, with capture-avoiding substitution. There are no reductions of formula constructors. Let $N(t)$ be the resulting normal form.

Lemma 3.1 (Iterator closure for strong normalization)**.** *Suppose n, a are strongly normalizing, and $h[v/u]$ is strongly normalizing for every strongly normalizing v. Then $I(n, a; u.h)$ is strongly normalizing.*

Proof. Let $\nu(r)$ be the maximum length of a reduction from a strongly normalizing term r. This maximum exists because the reduction tree is finitely branching. The body h is strongly normalizing by taking $v = u$. Use lexicographic induction on

$$(\nu(n),\ |n|,\ \nu(a) + \nu(h)).$$

A step in n lowers the first coordinate. A step in a or h lowers the last coordinate; in the latter case the universal substitution hypothesis persists because reduction is stable under substitution. The zero root reduct is a. For a successor counter Sp, the predecessor has the same maximum reduction length and smaller syntax size. The induction hypothesis makes $I(p, a; u.h)$ strongly normalizing, and the universal hypothesis on h then applies to the successor root reduct. Thus every one-step reduct is strongly normalizing. □

Induction on the original finite syntax now shows that replacing its free variables by strongly normalizing terms always produces a strongly normalizing term: the update-body induction hypothesis supplies the universal premise of the lemma. In particular, all finite raw terms normalize.

The reductions are locally confluent. Disjoint reductions commute, the zero and successor root patterns are disjoint, and an inner reduction overlapping a root contraction can be performed in all its finitely many residual copies after the contraction. Erased residuals need no step. Termination and local confluence imply unique normal forms. Consequently

$$N(t[\sigma]) = N(N(t)[x \mapsto N(\sigma(x))]). \qquad \text{(Coherence)}$$

Indeed, both sides are normal forms of convertible raw terms. This gives the composition and binder-lifting laws for normalized substitution. The same statements apply to formulas by recursion on their unchanged constructors.

3.2 Why normalization does not strengthen the arithmetic

Each root reduction is a computation equation of the raw scalar calculus. Each contextual reduction is provable there by congruence. Congruence in an iterator update follows by equation induction on the counter, with the initializer and passive parameters fixed. Thus the raw theory proves $t = N(t)$ for every particular finite term and proves the corresponding formula equivalence.

Conversely, normalize every term in a raw derivation and use normalized substitution for its instances. The two computation axioms become reflexivities. A normalization that changes a nonzero-spelled witness to the literal 0 can change the single introduction's selected mode. The

compound-envelope repair proved in the main construction supplies the missing case introduction. This is the inherited canonical/raw comparison; the argument is not a claim that every primitive proof tag survives normalization unchanged.

The canonical and raw presentations therefore have effective proof translations and formula round trips. Their numerical domains are the same. Their equality is the same actual equality. Building computations into syntax does not add a uniform numerical evaluator to either theory.

3.3 Uniform resource bounds

The termination proof is metasyntactic. It is not an axiom asserting normalization totality inside iPRA. The nesting depth of finite primitive recursive descriptions is unbounded, so one must not infer a uniform primitive-recursive bound on normalization from the primitive recursiveness of each separately fixed numerical function.

A resource-bounded verifier can instead accept a natural-number fuel value and charge every recursive verification or normalization operation against it. Every successful finite run has some sufficient fuel. This gives the same theorem set when the fuel is part of the certificate. No uniform sufficient-fuel function is assumed here. Resource limits of a particular Python process can reject an otherwise valid long proof; mathematical completeness has the same sufficient-resource convention as the comparison programs.

4 Full first-order iPRA and both interpretation composites

The comparison is with full first-order intuitionistic primitive recursive arithmetic, not just its equational fragment or its closed numerical consequences. This section gives the interpretation maps and identifies the inherited mathematical work needed for the reverse direction.

4.1 The target theory

Use ordinary intuitionistic first-order logic with actual equality and falsity. Its language has $0, S$ and a symbol for every fixed primitive recursive definition, generated from zero, successor, projections, composition, and primitive recursion. The nonlogical axioms are those definitions' equations, successor separation and injectivity,

$$Sx = 0 \to \bot, \qquad Sx = Sy \to x = y,$$

and induction for every quantifier-free formula $A(n, \vec{p})$:

$$\big(A(0, \vec{p}) \land \forall n(A(n, \vec{p}) \to A(Sn, \vec{p}))\big) \to \forall n A(n, \vec{p}).$$

The full logic admits the usual introduction and elimination rules for $\land, \lor, \to, \forall, \exists$, falsity elimination, assumption use, equality reflexivity, and equality substitution. Universal introduction and existential elimination have their ordinary fresh-variable conditions. Excluded middle for arbitrary formulas is not a rule. This is the theory deeply embedded as `PR` in the pinned repository.

4.2 An intermediate ordinary scalar theory

Let T_I be intuitionistic first-order equality logic on the scalar iterator terms, with the two computation equations, coverage,

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

separation $(0 = S0) \to \bot$, and contextual induction for literal equations. Elaborate a fixed update term $h(u, \vec{p})$ into an ordinary function symbol $i_h(n, a, \vec{p})$ when an orthodox first-order signature is desired. Its defining equations are

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

These declarations are staged by finite syntax. There is no numerical sort of arbitrary update programs and no universal evaluator.

The interpretation of native syntax into T_I leaves numerical terms and equations unchanged and sends

$$D_{x,y}B \quad \mapsto \quad \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

An unconditional clause becomes $\forall y B^\circ(y)$; a conditional clause becomes $A^\circ \to \forall y B^\circ(y)$. The fixed reflexivity clause is a theorem. The soundness of the new Open rule was proved directly using these readings.

The separation axiom in the ordinary realization must be stated. Positive computation equations alone also hold in a singleton structure. Native falsity being encoded by $0 = S0$ does not justify silently removing the ordinary falsity bridge.

4.3 All ordinary logical operations are available

Put $q_y A = D_{w,y} A$ for an unused w. Define the selector on equations by

$$\mathrm{Sel}(v, s = t, r = u) := C(v, s, r) = C(v, t, u),$$

and recursively on equally shaped paired formulas by moving the selector into their matrices, freshening bound variables. To combine formulas of different shapes, pad an equation E by the provably equivalent compound envelope $q_z\,\mathrm{Gate}(z, E)$ and then pad the shorter binder depth. Here

$$\mathrm{Gate}(v, s = t) := C(v, 0, s) = C(v, 0, t),$$

and gating also moves recursively through paired binders. Thus the zero gate is provable and its successor instances are equivalent to the input. The inherited native logical interface gives the following macros:

$$\begin{aligned}
\bot^* &:= 0 = S0,\\
A \to^* B &:= q_y\,\mathrm{Sel}(y, A, B),\\
A \wedge^* B &:= q_y\,\mathrm{Gate}(y, \mathrm{Sel}(P(y), A, B)),\\
\exists^* x A(x) &:= D_{x,y}\,\mathrm{Gate}(y, A(x)),\\
A \vee^* B &:= \exists^* v\,\mathrm{Sel}(v, A, B),\\
\forall^* x A(x) &:= A(0) \wedge^* q_x A(x).
\end{aligned}$$

All auxiliary variables are fresh. The padding is a syntactic construction, not a new axiom. Clause use and the recovered ordinary paired elimination give closure specialization. Numerical cases give

the selector endpoints; the compound envelope gives unrestricted case introduction. These recover every intuitionistic logical rule for the displayed macros. No induction on an arbitrary quantified formula is introduced by this logical recovery.

For example, under A a proof of B gives a proof of the successor instance of $\mathrm{Sel}(y, A, B)$ from its zero instance. Closure introduction gives $A \to^* B$. Closure specialization at 1 gives its elimination rule. For an existential, the gated zero instance is provable and the successor instance is the witnessed formula. For a conjunction, specialization at $1, 2$ selects its two components using the two parity endpoints.

4.4 The numerical interpretations

An iterator translates to the primitive recursive function defined by iteration with an update that ignores the recursion counter. iPRA proves its equations, equation induction, coverage, and separation.

In the opposite direction the inherited scalar development proves, in this order, a predecessor, reflection of actual equality by a numerical test, characteristic equations for all quantifier-free formulas, and pairing with projections. It is important to preserve that order: neither pairing nor a faithful equality detector is assumed in obtaining the predecessor.

If π, L, R are the resulting pairing and projection terms, then a primitive recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is translated by

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), R(u), \vec{p}))\big),$$
$$f^*(n, \vec{p}) = R(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$, and the two projection identities prove the defining equations for f^*. The construction recurses externally over each fixed finite primitive recursive description. It does not quantify over arbitrary functions internally.

4.5 Whole domains and return translations

The interpretation domain in both directions is defined by $x = x$. Interpreted equality is actual equality. Uniqueness of a fixed iteration follows by equation induction on the equality of two solutions with the same seed and update. The analogous argument applies to primitive recursion. External induction on each finite function description then proves that both numerical term composites equal the original term on the entire numerical domain. Both numerical composite isomorphisms have graph $x = y$, including at nonstandard elements.

The logical macros and the ordinary reading have formula round trips by structural induction. They cover arbitrarily nested quantifiers and arbitrary retained contexts. The new opening calculus is equivalent to the canonical reference on the entire clause consequence relation, so this existing bridge transfers without changing a domain or an equality. In particular, the unchanged contradiction $0 = S0$ gives equiconsistency.

These are the inherited bridge results formalized in the pinned `BiInterpretation`, `NativeClausal`, and `NativeBiInterpretation` modules.[4] Relevant endpoints include `biInterp`, `cprf_iff_nprf`, `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff`. The new deterministic-opening comparison and the full-normalization presentation remain written proofs; they are not additional claimed Lean endpoints.

5 The executable presentation and its checks

5.1 Exact certificate interface

The selected new checker is `code/checker_open.py`. It is a standalone standard-library Python module. Its input is one JSON object with exactly the fields `goal` and `proof`. The goal must be a closed canonical formula. Successful checking prints `ACCEPT` and exits with status zero. Exceptions print `REJECT` and exit nonzero. Trust decisions do not use Python `assert`, so optimization does not remove them.

The sorted encoding uses integer 0 for numerical zero, negative integers for free parameter levels, and positive integers for bound de Bruijn indices. A term successor is `[0,t]`, an iterator is `[n,a,h]`, an equation is `[s,t]`, and a paired formula is `[B]`. The same two-element raw list can therefore encode a successor or an equation; the syntactic sort distinguishes them. An iterator binds only its update body. A paired formula binds two variables, with y the innermost coordinate and x the next coordinate.

At numerical scope n, fresh parameter b is represented by $-n-1$. Clause positions are nonnegative actual integers, not Boolean values. The initial context is the single unconditional reflexivity clause. The three proof packets are

```
["u", clause_index, term, antecedent_proof]
["i", witness, branch_proof]
["e", matrix, major_proof, branch_proof]
```

An unconditional clause has no antecedent proof; that payload is ignored, and `null` is the recommended spelling. This is a nullary inference, not acceptance of an unchecked premise. The Open packet is shorter than its predecessor's

```
["e", matrix, branch_formula, target,
      major_proof, equality_proof, branch_proof]
```

The shared syntax walker checks sorts, scope, and canonical normal form. It also implements capture-free normalized substitution. Open first checks its major, then probes the matrix at fresh b and at numerical 1, chooses the witness by the stated literal test, and adds the clause at that chosen value. Its branch is checked against the same outer goal, in scope $n+1$. The outer goal has already been checked in scope n, which enforces the eigenvariable condition even when the chosen value contains b.

Because the probe is already known to be a well-formed formula, testing whether its first child is the integer representing b also establishes that it is an equation: a D formula has a formula, hence a list, as its first child. This implementation fact is not permission to omit the mathematical requirement that the probe be literally $b = t$.

The zero-row test also uses the checked grammar: `not any(base)` is true exactly for the equation `[0,0]`. Every nonzero numerical term is a nonzero integer or a nonempty list; the sole child of a D formula is a nonempty formula list. No other well-formed formula has only false-valued immediate children. The major proof is checked before this test, so the invariant is already established. After the successful left-child test, the equation's right side is safely selected at index `1`.

5.2 Measured effect of the new rule

The old and new complete modules share the same normalizer, syntax checks, Use, Introduce, JSON entry point, and verdict convention. The changed instruction is Open. After removing actual docstrings and applying `ast.unparse`, with one terminating line feed, the sizes are

	bytes	significant tokens	AST nodes
predecessor	2273	732	813
automatic opening	2253	730	809

The measurement is a modest reduction in all three matched source-size coordinates. It does not approach the historical compact protocol's byte count; the two interfaces use substantially different encodings and validation contracts. The complete selected source is reproduced in Appendix A and supplied separately for execution.

For completeness the archive retains an intermediate variant that determines the same equality but substitutes in a supplied branch formula at export. It has three instruction families and two proof premises at discharge, but still requires the intermediate formula. Its checker is 2,296 bytes, 745 tokens, and 828 AST nodes, so it is not the selected implementation. Its proof and tests document the route to the stronger opening rule; its size is not reported as a record.

5.3 Executed evidence

The primary opening suite has 16 accepted and 19 rejected closed certificates. Each was run in a fresh ordinary Python process and a fresh optimized process. The 16 accepted theorems also have matching proofs checked by the old checker and independently replayed by a differently structured inherited kernel. The suite additionally checks a genuinely open parameter sequent. Its examples include symmetry, transitivity, repeated occurrences in a numerical template, nested iterator congruence, nested formulas, a used induction hypothesis, default openings, forced witness selection, and a selected term containing the fresh witness.

The difficult substitution transition also has a direct test. A source proof whose opening uses the fresh parameter is accepted. Substituting only its passive parameter makes extraction succeed and leaves an invalid branch, which is rejected. Instantiating the original smaller branch at the newly selected witness, exactly as in the substitution theorem, is accepted. The same three results hold with Python optimization enabled.

The intermediate variant adds 17 accepted and 20 rejected fixtures and a checked executable translation of its valid fixtures to the predecessor. Together the two suites and their guard mutants use 222 fresh checker processes, including optimized runs. All expected verdicts hold. The independent source measurements match the table above.

Deliberately removed condition	Result of the attack certificate
Probe's left side must be the fresh witness	Mutant accepts $1 = 0$.
The normalized zero row must be $0 = 0$	Mutant accepts $1 = 0$.
Outer goal must have the outer numerical scope	Mutant accepts an escaping fresh parameter.

The correct checker rejects all three attacks. These tests matter because the changed instruction's apparent simplicity depends on those exact conditions; removing them is not another valid simplification.

For the primary opening system, the executable tests supply matching old proofs of the tested theorems. They do not constitute a generic executable compiler for every opening proof. The universal translations, including the strong-substitution construction and its introduction-mode repair, are the written metatheorems above. The separate compact refactor's tests and trace proof are reported in the next section.

5.4 Reproduction

The archive has a single top-level reproduction command:

```
python3 reproduce.py
```

It reruns both new-rule suites, the compact matched comparisons, and the source measurements. To check an individual new proof, for example:

```
python3 code/checker_open.py < examples_open/positive/reflexivity_0.json
```

The bundled `build_report.py` assembles a standalone LaTeX source from the editable sections and embeds the selected source listings. Run it with `-pdf` to compile the PDF. LaTeX compilation requires `pdflatex` and standard TeX Live packages; the mathematical certificate programs require only Python's standard library.

6 A separate, measured implementation reduction

This section concerns the existing scalar proof protocol, independently of the new mathematical presentation. The comparison sources are frozen at the commit stated in Section 1; their exact Git blob hashes appear in `compact/PROVENANCE.json`. The implementations count the entire acceptance module after `ast.unparse` under CPython 3.12.14, followed by one newline.

Profile	Tokens	AST nodes	Bytes	Gzip-9
Compact control	265	306	516	332
Compact fused	267	302	508	330
AST control	260	278	602	344
AST fused	263	276	580	343
Native validated control	358	413	788	460
Native validated fused	360	409	780	458

Thus the observed byte coordinate decreases from 516 to 508, and the observed AST coordinate from 278 to 276, in different programs. The separate token leader remains 256 tokens. No single program in this table improves every coordinate, and these are attained sizes, not universal lower bounds.

The compact and AST profiles retain the upstream closed guarded Eq/D public-goal contract and integer/list certificate language. They do not independently validate the public-goal grammar.

The native profile retains the validator of its matched upstream control. Its 780 bytes must not be confused with the 508-byte compact interface. None of these figures is a measurement of the new deterministic-opening theory.

The transformation. Let I_k denote scoped opening, and let R be the optional elimination rewrite of the current goal. The old checker rewrites the goal, constructs a new hypothesis, decrements the scope floor k, then opens the continuation. The new checker constructs the hypothesis first and fuses rewriting with the continuation call. Its essential expression is

```
c(i(rewrite(goal), [(k := k - 1)]), new_context, k)
```

Python evaluates argument expressions from left to right before invoking the called function. Consequently the rewrite still uses the old scope, the second argument allocates the fresh parameter, and the opening itself uses the new scope. This relies on the ordinary evaluation-order rule, not an implementation-specific optimization. See the Python language reference, Section 6.16, at https://docs.python.org/3.12/reference/expressions.html#evaluation-order.

Why the reordering preserves proofs. The expected elimination major is either the nonempty equality list $[s, u]$ or the nonempty closure list $[t, t, t]$. A rewrite can therefore change the goal only when the major m is nonempty. In that case the hypothesis-construction expression stops at $m * 3$ and cannot perform the introduction assignment. Conversely, if introduction changes the goal, m is empty and the rewrite is the identity. These operations commute.

The major subproof uses a separate function activation with the same scope and context; its local scope changes cannot affect the caller. The rewrite does not consume proof packets. Moving it past the major subproof therefore preserves successful results and the remaining packet suffix. Failure may occur in a different order, which does not change acceptance.

The context's mode selector also agrees. With a nonempty major it does not inspect the goal; with an empty major the moved rewrite is the identity. In the iterative AST profile, opening preserves the root arity of every successfully opened list. A scalar case still rejects wherever the old arity test rejected. Finally, the AST profile preserves a false rewritten value with a conditional expression, whereas the compact profile retains its old Boolean fallback literally.

Induction on the finite number of available packets now proves that each new program accepts exactly the certificates accepted by its matched control, consuming the same suffix on successful runs. This statement uses the documented integer/list interface and the usual sufficient-resource convention. It immediately transfers the inherited logical consequence relation, interpretation maps, actual equality and identity composites. It is a written program-equivalence proof, not a new proof-assistant verification or a mathematical axiom deletion.

Executable verification. The archive contains authenticated positive and negative fixtures, all controls, the new standalone modules, and a reproduction script. Verification comprised 252 standalone executions with and without Python optimization, 500 deterministic certificate mutations in three matched pairs, and a fresh-parameter boundary check in each pair. All 1629 paired comparisons agreed. These finite checks test the implementations; the argument above establishes the universal correspondence.

7 Further research questions

1. Can the fixed-probe opening rule be represented in the smallest arity-based certificate language without losing the source savings to normalization and scope bookkeeping? The present 508-byte program belongs to the inherited calculus, not to this new rule basis.
2. Can the entire substitution theorem for deterministic opening be formalized directly in Lean? The interesting case is a substitution that creates an extractable equality where the old opening used a fresh parameter. Its branch must be instantiated at the newly determined term.
3. Can a suitably designed canonical syntax expose both the zero row and first row without full normalization, while retaining all current formula round trips? A syntactic shortcut must justify its whole-domain interpretation and count its decoder.
4. Is there a complete presentation improving the token, AST, and byte coordinates simultaneously under one fixed input contract? The present coordinate leaders remain different programs.
5. Can proof translations between the new presentation and the raw native core be given as fully checked certificate compilers, including normalization traces and the introduction-mode repair? The current universal result is a written metatheorem, not a claimed general executable reverse compiler.
6. What are useful lower bounds under a genuinely fixed implementation language and public theorem interface? The observed corpus minima alone provide no such lower bounds.

References

[1] V. Reshetnikov, ProveIt, *Concise presentations of intuitionistic primitive recursive arithmetic*, README and research inventory at commit `17898ea7e6a1300dfdcad71040d5daeed67916d4`. Pinned project directory.

[2] *Witness discharge*, incoming research archive `ipra_witness_discharge_2026-11-11.zip`, in the same pinned project, `docs/incoming`. Git blob `3e2c86ca3590588ea3be3ebf24dd379a3a660ff1`. The immediate control and supporting mathematical sources are reproduced in the accompanying archive with provenance.

[3] *Full normalization*, research package in `Research/full_normalization` of the pinned project. Written termination, confluence, coherence, and canonical/raw comparison.

[4] ProveIt Lean development, `NativeClausal`, `NativeBiInterpretation`, and `BiInterpretation`, at the same pinned commit. Selected exact source files are included for reference; this report does not claim a new build or new Lean theorem.

[5] Python Software Foundation, *Python 3.12 Language Reference*, Section 6.16, Evaluation order. `https://docs.python.org/3.12/reference/expressions.html#evaluation-order`.

A Complete selected checker

This listing is the entire executable module apart from its informational header docstring. The separately supplied file contains that header. Comments are not counted by the source normalization

metric.

```
import json
import sys


def need(condition):
    if not condition:
        raise ValueError('invalid certificate')


# With a Boolean form, the same traversal validates syntax before reducing.
# Generic traversals receive only checked syntax or constructed iterators.
def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or size in ((1, 2) if form else (2, 3))
            and (form or size == 3 or a[0] == 0))
    a = [walk(b, f, d + (2 if size == 1 else int(size == 3 and i == 2)), size == 1)
         for i, b in enumerate(a)]
    if size == 3:
        c, b, h = a
        if c == 0:
            return b
        if type(c) is list and len(c) == 2:
            # Normalize the predecessor iterate before capture-avoiding insertion.
            return sub(h, walk([c[1], b, h], lambda i, d: i))
    return a


def syntax(a, n, form=False):
    def variable(i, d):
        need(-n <= i <= d)
        return i
    need(walk(a, variable, form=form) == a)


def shift(a, k):
    return walk(a, lambda i, d: i + k if i > d else i)


def sub(a, t, depth=0):
    return walk(a, lambda i, d: shift(t, d) if i == d + 1 else i - (i > d + 1), depth)


def prove(a, p, g, n):
    syntax(a, n, True)
    match p:
        case ['u', j, t, premise]:
            need(type(j) is int and 0 <= j < len(g))
            syntax(t, n)
            base, body = g[j]
            need(a == sub(body, t))
            # Empty antecedent means no logical premise. The payload is unused.
            if base:
```

```
                prove(base, premise, g, n)
        case ['e', body, major, branch]:
            prove([body], major, g, n)
            opened = sub(body, -n - 1, 1)
            row = sub(opened, [0, 0])
            # In the checked formula grammar, only 0=0 has all false children.
            # A first child equal to the fresh integer also ensures equation arity.
            body = sub(body, row[1] if not any(sub(opened, 0)) and row[0] == -n - 1 else -n - 1,
    1)
            prove(a, branch, g + [(sub(body, 0), body)], n + 1)
        case ['i', t, branch]:
            need(len(a) == 1)
            syntax(t, n)
            body = sub(a[0], t, 1)
            hyp = sub(body, -n - 1 if len(body) == 2 and t == 0 else 0)
            prove(sub(body, [0, -n - 1]), branch, g + [([], hyp)], n + 1)
        case _:
            raise ValueError('invalid proof')


def check(data):
    need(type(data) is dict and set(data) == {'goal', 'proof'})
    prove(data['goal'], data['proof'], [([], [1, 1])], 0)


if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

B The separate 508-byte compact profile

The following module implements the inherited compact protocol under its documented guarded public-goal contract. It is not the selected checker of the new three-rule theory.

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a <
     d else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a
    := i(*a, t, 2))
    c(i((m, a) == (s and [s, u] or [t] * 3, i(t, u)) and i(t, s) or a, [(k := (k - 1))]), g + [i(
    h, (m or ~len(a) % 2) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```